

EnvMonitor 环境监控上位机 Demo

详细设计文档（基于当前代码反向推导）

文档类型：代码反向设计文档

目标框架：.NET 8 / WPF

系统定位：无真实下位机条件下的 TCP 环境设备模拟与上位机演示系统

覆盖版本：当前 main 分支代码

验证结果：30/30 测试通过，0 个错误

1. 文档范围

本文档根据当前仓库中的解决方案、项目文件、协议实现、TCP 服务、WPF ViewModel 和测试代码反向整理而成，描述当前代码实际行为，不把尚未实现的功能当作已完成能力。

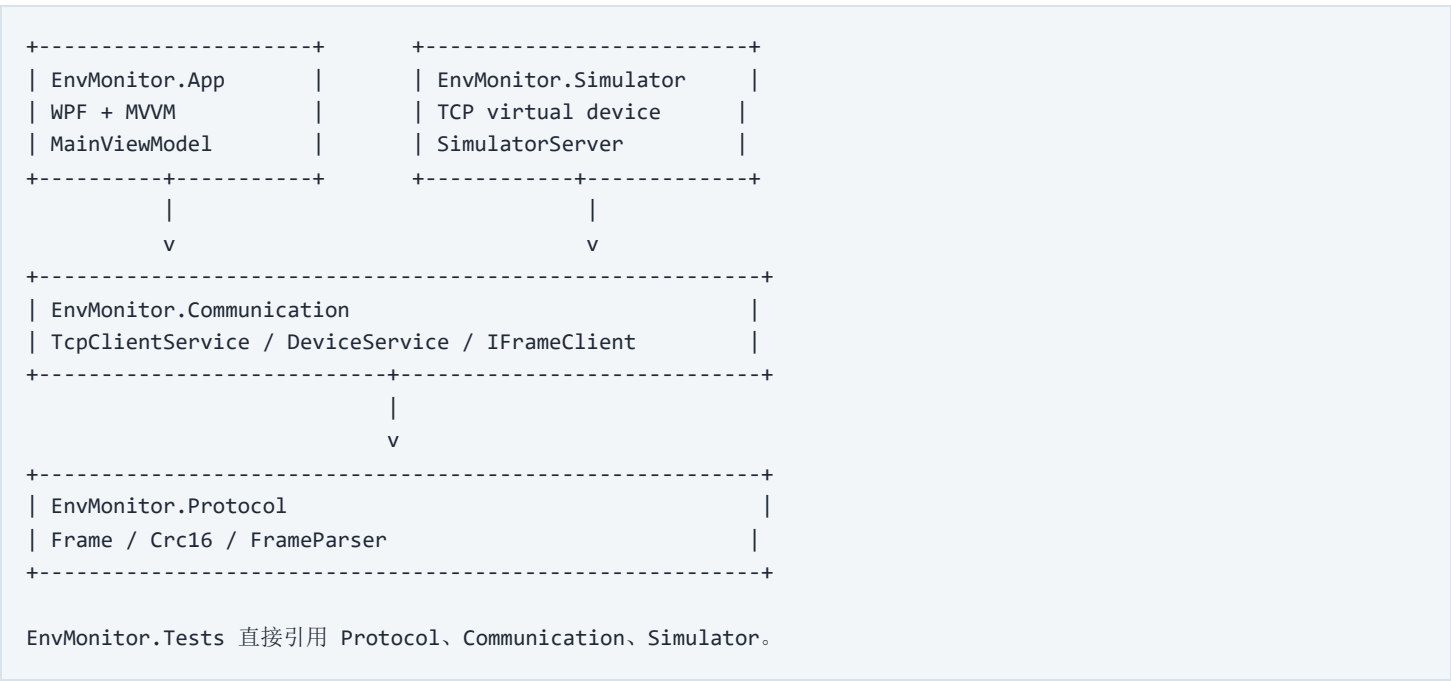
- 解决方案与项目边界
- TCP 二进制协议和 CRC16
- 帧拆包与错误恢复
- 模拟下位机和故障注入
- 上位机客户端、请求匹配、超时与重连
- 业务服务、WPF MVVM、轮询、报警和曲线
- 配置、日志、构建和测试

2. 系统目标

系统模拟一个环境传感器下位机，通过 TCP 让 WPF 上位机完成连接、命令交互、数据刷新和异常处理。

- 建立 TCP 连接。
- 发送读取、继电器和心跳命令。
- 接收并解析二进制帧。
- 处理半包、粘包、CRC 错误、超时和断线。
- 以 MVVM 方式显示传感器数据。
- 对温度、湿度超阈值进行状态报警。
- 显示最近两分钟温度曲线。

3. 总体架构



依赖方向：Communication -> Protocol，Simulator -> Protocol，App -> Communication，Tests 引用三层用于集成测试。Protocol 不依赖 TCP、WPF 或业务层。

4. 项目职责

项目	核心类型	职责
Protocol	Frame、Crc16、FrameParser	帧格式、CRC、拆包和错误恢复。
Communication	IFrameClient、TcpClientService、DeviceService、SensorData	TCP 可靠请求/响应、重连和强类型业务 API。
Simulator	SimulatorServer、VirtualDevice、FaultOptions	虚拟设备、动态数据和确定性故障注入。
App	App、MainViewModel、MainWindow	配置、日志、MVVM、轮询、报警和曲线。
Tests	xUnit 测试集	协议、通信、业务和异常场景验证。

5. 协议详细设计

偏移	长度	字段
0	1	Header1 = 0xAA
1	1	Header2 = 0x55
2	2	Length, 大端序
4	1	Command
5	2	Sequence, 大端序

7	N	Payload
7 + N	2	CRC16，低字节在前

- 总长范围为 9 到 1024。
- CRC 覆盖 Header 到 Payload，不包含 CRC 本身。
- CRC 使用 Modbus 算法，初始值 0xFFFF，多项式 0xA001。
- 正常响应复用请求 Command 和 Sequence。
- 错误响应 Command 为请求 Command 按位或 0x80，Payload 为 1 字节 ErrorCode。

空读帧固定测试向量：

AA 55 00 09 01 00 01 8E 93

命令	请求 Payload	成功响应 Payload
0x01 Read	空	Temp Int16 BE、Humidity UInt16 BE、Pressure UInt16 BE，均除以 10，加 Relay Byte，共 7B
0x02 SetRelay	Channel Byte、On Byte	Channel Byte、On Byte
0x03 Heartbeat	空	空

错误码：0x01 非法命令，0x02 非法 Payload，0x03 非法继电器通道。

6. 帧拆包设计

FrameParser 使用 List<byte> 缓存。Feed 流程是：追加数据、搜索 AA 55、丢弃前置垃圾、读取长度、等待完整帧、调用 Frame.Decode、成功后移除完整帧，失败时只丢弃一个候选帧头字节。

因此支持帧头拆分、完整帧拆分、多帧合并、CRC 错帧后恢复和非法长度后恢复。当前实现适合学习和演示，高吞吐场景可替换为环形缓冲区或 PipeReader。

7. 模拟器设计

7.1 生命周期

StartAsync() 创建 Loopback TcpListener；端口 0 由系统分配并暴露实际端口。每个客户端拥有独立 FrameParser 和处理任务。StopAsync() 取消服务器 CTS、停止 Listener、关闭客户端并等待 Accept 循环。

7.2 虚拟设备

VirtualDevice 在锁内处理请求，维护温度、湿度、压力和继电器状态。传感器初始值约为 25.0°C、48.0%、101.3kPa；Read 命令使用 Random.Shared 做小幅波动；继电器当前只支持 Channel 1。

7.3 故障注入

选项	行为
DropProbabilityPercent	按概率不回复。
ResponseDelay	异步延迟响应。
CloseAfter	连接建立后定时关闭。
CrcErrorResponseNumber	指定响应损坏 CRC。
FragmentResponses	一帧拆成两次写入。
CoalesceResponses	同一批次多帧合并写入。

8. 上位机通信设计

8.1 请求发送

`SendAsync()` 原子生成 Seq，创建带 `RunContinuationsAsynchronously` 的 TCS，放入 Pending 字典；使用 SemaphoreSlim 串行写流；等待响应或超时；在超时、取消或写失败时清理 Pending。

8.2 接收循环

NetworkStream 的每次读取都送入 FrameParser。Seq 匹配 Pending 时完成对应 TCS，否则触发 UnsolicitedFrame。CRC 错误由解析器丢弃，原请求最终超时。读到 0 或异常会完成未决请求并进入断线处理。

8.3 状态与重连

连接状态包括 Connecting、Connected、Disconnected、Reconnecting。意外断线启动唯一重连循环，等待 1、2、4、8 秒，最大 30 秒；重连使用新的 TcpClient、Stream 和 CTS。手动 DisconnectAsync 会取消重连并完成未决请求。

9. 业务服务设计

DeviceService 隔离 Frame 细节：ReadAsync 要求 7 字节并进行大端缩放；SetRelayAsync 检查 Channel 和状态回显；HeartbeatAsync 要求空响应。错误响应转换为 InvalidOperationException，非法长度或字段转换为 InvalidDataException。

10. WPF 与 MVVM

App 启动时加载 appsettings.json、校验配置、初始化 Serilog，并组装 TcpClientService、DeviceService、MainViewModel。MainWindow 只通过 DataContext 绑定 ViewModel。

MainViewModel 暴露状态、连接按钮、温湿压、继电器、报警状态、曲线数据以及 Connect、Reconnect、ToggleRelay 命令。DispatcherTimer 按配置周期轮询，使用 `_isPolling` 防止重入；断开时停止 Timer 并清空曲线。

温度超过配置阈值 30、湿度超过 70 时状态变为 Alarm，XAML DataTrigger 将对应数值变红；读取超时显示黄色状态。曲线保留最近 120 个 DateTimePoint。

11. 配置与日志

```
{
  "Device": {
    "Host": "127.0.0.1",
    "Port": 9000,
    "PollIntervalMs": 1000,
    "RequestTimeoutMs": 1000
  },
  "Thresholds": { "Temperature": 30, "Humidity": 70 }
}
```

配置通过 PreserveNewest 复制到输出目录。Serilog 使用按天滚动文件，保留 14 个文件，路径为运行目录下的 logs/app-日期.log。当前记录应用启动、连接、状态变化、轮询、超时、继电器和异常。

12. 测试与结果

测试层	覆盖内容
Protocol	CRC16、帧字节序、编解码、半包、粘包、垃圾和 CRC 恢复。
Simulator	Read、SetRelay、Heartbeat、动态数据和错误帧。
Communication	并发请求、Seq 匹配、超时、Pending 清理和自动重连。
DeviceService	大端解析、缩放、Payload 校验和错误响应。
Fault injection	丢帧、延迟、CRC 错、fragment、coalesce 和非法命令。

运行 `./build.ps1` 可执行 Release 构建和测试。当前结果为 30/30 通过、0 错误。LiveCharts 传递依赖仍产生 3 类 NU1701 警告：OpenTK、OpenTK.GLWpfControl、SkiaSharp.Views.WPF。

13. 运行与演示

```
dotnet run --project EnvMonitor.Simulator -- --port 9000
dotnet run --project EnvMonitor.App
```

1. 启动模拟器和 WPF App。
2. 点击 Connect，观察每秒数据刷新。
3. 切换继电器并观察状态。
4. 观察最近两分钟温度曲线。
5. 使用 `--close-after 10` 验证重连。
6. 使用 `--drop 30` 或 `--delay 2000` 验证超时。
7. 修改阈值或数据范围观察报警。

14. 已知限制与风险

发布前需要关注：LiveCharts 依赖链存在 NU1701；发布前应确认目标机器运行时兼容性，或升级/替换图表包。

- FrameParser 使用 `List<byte>`，高吞吐场景会有额外复制。
- TCP 客户端没有 TLS、认证或设备身份校验。
- WPF ViewModel 没有独立测试项目，主要依靠集成测试和构建验证。
- 日志使用 App 层 Serilog，Communication 层没有独立 ILogger 抽象。
- 模拟器随机波动只用于演示，不代表真实传感器噪声。

15. 发布检查清单

- ☐ Release build 成功
- ☐ 全部测试通过
- ☐ 在干净目录启动模拟器和 WPF App
- ☐ 修改端口配置后重启验证
- ☐ 验证日志生成
- ☐ 验证断线重连、超时和报警
- ☐ 录制连接、刷新、曲线、继电器、重连、报警演示
- ☐ 确认 NU1701 的发布结论
- ☐ 再创建 v1.0 标签